

PashuPehchaan

Image Based Breed Recognition System
for Cattle and Buffaloes of India

Complete Technical Project Documentation
How the System Works — End to End

Project Type	Web-based AI Application (Localhost)
Domain	Agriculture / Livestock Management
Frontend	HTML, CSS, JavaScript
Backend	Python FastAPI
AI Model	ConvNeXt-Tiny (Indian Bovine Breeds)
Degree	Bachelor of Computer Applications (BCA)
Institution	Vidya Vihar Institute of Technology, Purnea, Bihar
University	Aryabhatta Knowledge University
Session	2023–2026
Document Date	28 July 2026

Submitted for partial fulfilment of the requirement for the award of the degree of
Bachelor of Computer Applications

Table of Contents

1. Introduction and Problem Statement
2. Project Objectives
3. Scope of the Project
4. System Overview — How It Works
5. Technology Stack
6. System Architecture
7. Project Folder Structure
8. Module-wise Explanation
9. Artificial Intelligence Model (Detailed)
10. End-to-End User Flow
11. API Reference
12. Frontend Design and User Interface
13. Backend Server and Startup Process
14. Breed Information Database
15. How to Run the Project
16. Testing and Expected Outcomes
17. Limitations
18. Future Enhancements
19. Conclusion

1. Introduction and Problem Statement

PashuPehchaan is a web-based Artificial Intelligence application designed to identify the breed of cattle and buffaloes found in India using a single uploaded photograph. In India, breed identification is still often performed manually by field workers, farmers, or livestock inspectors. Manual identification depends heavily on personal experience and can lead to incorrect classification when expertise is limited.

Incorrect breed records affect milk-yield planning, government livestock schemes, research data quality, and breeding programmes. Therefore, there is a clear need for an automated, fast, and user-friendly system that can recognise breeds from images using deep learning and computer vision.

This project implements that solution as a **localhost website** that runs on both macOS and Windows. The user opens the site in a browser, uploads a photo, and receives the predicted breed name with confidence score, region, milk yield, lifespan, usage, and alternative likely matches.

2. Project Objectives

- To develop an AI-based system for automatic breed identification of Indian cattle and buffaloes.
- To allow users to upload animal images through a clean, professional web interface.
- To provide accurate breed prediction using a trained deep-learning vision model.
- To display useful breed details such as milk yield, lifespan, region, and usage.
- To reduce manual errors in livestock data collection.
- To keep the system simple to start using a single command (`npm run dev`).

3. Scope of the Project

The system is intended for farmers, students, researchers, and government field staff who need quick breed identification. The current version runs offline on a local computer after the AI model is downloaded once. It focuses on image-based recognition and breed information display. Mobile apps, disease detection, and cloud deployment are outside the present scope but are listed as future work.

4. System Overview — How It Works

At a high level, PashuPehchaan works as a closed loop between the browser and a local Python server:

- **Step 1 — Start:** The developer/user runs `npm run dev`. Node.js launches the Python virtual environment and starts the FastAPI server with Uvicorn on `http://127.0.0.1:8000`.
- **Step 2 — Model load:** When the server starts, it loads a ConvNeXt-Tiny neural network trained on Indian bovine breeds (about 320 MB). The model stays in memory for fast predictions.
- **Step 3 — Open UI:** The browser loads the landing page with brand, hero image, upload box, and breed library.
- **Step 4 — Upload:** The user drops or selects a JPG/PNG/WEBP photo of a cow or buffalo.
- **Step 5 — Predict:** JavaScript sends the file to `POST /api/predict`. The backend resizes and normalises the image, runs it through the neural network, and converts output scores into breed probabilities.
- **Step 6 — Enrich:** The top breed name is matched with a local breed information database (region, milk yield, lifespan, usage, description).
- **Step 7 — Result:** The UI shows the breed, confidence bar, details, uploaded photo, and the next three alternative matches.

In simple words: **Photo** → **AI Model** → **Breed Name + Details** → **Screen**.

5. Technology Stack

Layer	Technology	Role
Frontend	HTML5, CSS3, JavaScript	User interface, upload, results display
Backend	Python 3, FastAPI, Uvicorn	REST API, file handling, prediction service
AI / ML	PyTorch, timm, torchvision	Load and run ConvNeXt-Tiny breed classifier
Model Source	Hugging Face Hub	<code>ujjwal75/indian-bovine-breeds-model</code>
Image IO	Pillow (PIL)	Open, resize, and convert uploaded images
Launcher	Node.js (npm scripts)	One-command setup and server start
Data	Python dictionary (breeds.py)	Breed metadata for the UI

6. System Architecture

The architecture follows a classic **client-server** pattern with an embedded AI inference layer:

Component	Description
Browser Client	Renders UI from static files. Captures image upload and calls APIs using Fetch.
FastAPI Server	Serves HTML/CSS/JS and exposes JSON APIs for breeds and prediction.
Classifier Module	Loads ConvNeXt weights once, runs inference for each uploaded image.
Breed Knowledge Base	Maps predicted labels to human-readable livestock information.
Model Files	Stored under models/ (weights ~320MB + classes.json).

Request path: Browser → FastAPI (/api/predict) → classifier.predict_breed() → ConvNeXt forward pass → Softmax probabilities → Top breed + alternatives → breeds.get_breed_info() → JSON response → Browser result panel.

7. Project Folder Structure

```
agri/
├── app/
│   ├── main.py          # FastAPI routes, startup, static hosting
│   ├── classifier.py    # AI model load + prediction
│   ├── breeds.py        # Breed metadata database
│   └── __init__.py
├── static/
│   ├── index.html      # Main web page
│   ├── styles.css      # Visual design
│   ├── app.js          # Frontend logic
│   └── hero-cow.jpg     # Landing page hero image
├── models/
│   ├── Indian_bovine_finetuned_model.pth
│   └── classes.json
├── scripts/            # npm run dev / setup helpers
├── package.json
├── requirements.txt
└── README.md
```

8. Module-wise Explanation

8.1 Image Upload Module (Frontend)

Implemented in `static/index.html` and `static/app.js`. The user can drag-and-drop an image or browse from disk. Supported formats are JPG, PNG, WEBP, and BMP, with a maximum size of 8 MB. After selection, a thumbnail preview appears with Identify and Clear buttons. The Clear button resets the form. On Identify, the file is packed into multipart form-data and posted to the backend.

8.2 Breed Prediction Module (AI)

Implemented in `app/classifier.py`. This is the core intelligence of the project. It loads the ConvNeXt-Tiny checkpoint, preprocesses each image to 224×224 with ImageNet normalisation, runs a forward pass, applies softmax, and ranks all 41 class scores. The highest score becomes the predicted breed; the next three become alternatives.

8.3 Breed Information Module

Implemented in `app/breeds.py`. After the AI returns a label such as *Gir* or *Murrah*, this module supplies region, milk yield, lifespan, usage, and a short description. Labels from the model (for example `Red_Sindhi`) are normalised to display names (Red Sindhi). If a predicted breed is not in the curated library, the system still returns a generic information card so the UI never breaks.

8.4 User Interface Module

The UI uses a unified soft-green theme. The landing section shows the brand name **PashuPehchaan**, a short headline, upload area, and a full-bleed cute cow hero image. Below that, a breed library lists supported indigenous breeds with filters (All / Cattle / Buffalo). Results appear in a smooth panel with confidence animation. No incorrect stock breed photos are shown in the library (text cards only), keeping the interface clean and trustworthy.

8.5 Application / Server Module

Implemented in `app/main.py`. FastAPI mounts static files, serves the homepage, and exposes `health/breeds/predict` endpoints. On startup (lifespan event), the AI model is preloaded so the first user prediction is faster. Invalid files and empty uploads are rejected with clear HTTP errors.

9. Artificial Intelligence Model (Detailed)

Earlier prototypes used colour heuristics and CLIP zero-shot matching. Those approaches often misclassified animals (for example, repeatedly predicting Deoni). The final system therefore uses a **dedicated Indian bovine breed classifier** published on Hugging Face:

Repository: `ujjwal75/indian-bovine-breeds-model`

Architecture: ConvNeXt-Tiny (via the `timm` library)

Classes: 41 cattle and buffalo breeds

Input size: 224 × 224 RGB

Weights file: `Indian_bovine_finetuned_model.pth` (~320 MB)

Class list: `classes.json`

9.1 Inference pipeline inside `classifier.py`

- Open uploaded bytes with Pillow and convert to RGB.

- Optionally downscale very large images (max side 1024) for speed.
- Resize to 224×224, convert to tensor, normalise with ImageNet mean/std.
- Run model forward pass on CPU or Apple MPS if available.
- Convert logits to probabilities with Softmax.
- Sort classes by probability; pick top-1 and top-2..4 alternatives.
- Normalise label names and attach livestock metadata.

Because the network was trained specifically on bovine breed images, it understands coat patterns, body shape, and breed-typical appearance much better than colour-only rules. This is why accuracy improved after switching to the ConvNeXt model.

9.2 Example breeds the model can recognise

Gir, Sahiwal, Red Sindhi, Tharparkar, Hariana, Kankrej, Ongole, Hallikar, Rath, Deoni, Murrah, Jaffarabadi, Mehsana, Surti, Nili-Ravi, Bhadawari, Jersey, Holstein Friesian, Vechur, Kangayam, and many more (41 total).

10. End-to-End User Flow

#	User Action	System Response
1	Runs npm run dev	Creates/uses Python venv, starts Uvicorn on port 8000
2	Opens browser URL	Loads landing page + hero image + breed library
3	Uploads animal photo	Shows thumbnail preview and Identify button
4	Clicks Identify breed	Button shows "Analyzing with AI..."; POST /api/predict
5	Waits 1–3 seconds	Model computes breed probabilities
6	Views result panel	Sees breed, confidence, facts, alternatives
7	Optional: Clear	Resets upload state for a new photo

11. API Reference

Method	Endpoint	Purpose
GET	/	Serve the main web application page
GET	/api/health	Report whether the AI model is loaded
GET	/api/breeds	List curated breeds for the library UI
GET	/api/breeds/{name}	Return details for one breed
POST	/api/predict	Accept image file field "file"; return prediction JSON

Sample prediction response fields: breed, confidence, animal_type, method (convnext_bovine), details (region, milk_yield, lifespan, usage, description), and alternatives[{breed, confidence, type}].

12. Frontend Design and User Interface

The interface is intentionally clean and professional. Design choices include:

- One consistent soft-green colour theme across hero, library, and footer (no dark/light split).
- Brand-first landing: “PashuPehchaan” is the dominant text signal.
- Full-bleed cute cow hero photograph with a soft gradient so text stays readable.
- Typography: Instrument Serif for display headings, Manrope for body text.
- Smooth animations for reveal, confidence bar, card entrance, and loading spinner.
- Breed library as text cards (no unreliable breed photos) with All/Cattle/Buffalo filters.
- Responsive layout for desktop and mobile widths.

13. Backend Server and Startup Process

The project is started with Node for convenience, while the real application is Python:

- `npm run setup` — creates `.env` (if needed) and installs `requirements.txt`.
- `npm run dev` — ensures environment is ready, then runs `uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`.
- Cross-platform helper scripts detect Windows vs macOS/Linux Python paths inside `.env`.
- FastAPI lifespan hook calls `classifier.preload()` so ConvNeXt weights are loaded at boot.
- Static assets are served from `/static`; the homepage is `FileResponse` of `index.html`.

14. Breed Information Database

The curated library shown on the website focuses on widely known Indian breeds. Each entry stores type (Cattle/Buffalo), region, milk yield per lactation, lifespan, usage, and a short description. Examples include Gir and Sahiwal (cattle) and Murrah and Nili-Ravi (buffalo). The AI model can also return additional breeds outside this curated list; those are still displayed using generated basic details.

15. How to Run the Project

Requirements

- Computer/Laptop with at least 8 GB RAM recommended
- Python 3.9+ installed
- Node.js and npm installed
- Internet only needed once to download Python packages / model (if not already present)

Commands

```
cd agri
npm run setup    # optional first-time setup
npm run dev
# then open http://127.0.0.1:8000
```

16. Testing and Expected Outcomes

Expected behaviour after correct installation:

- Homepage loads with brand, hero cow image, and upload box.
- Uploading a clear side/front photo of cattle or buffalo returns a breed prediction.
- Confidence percentage and alternative matches are shown.
- Breed library filters work without page reload.
- Health API reports `model_ready: true` after startup.
- Invalid non-image files are rejected with an error message.

The expected academic outcome is a working demonstration that image-based deep learning can support livestock breed identification and reduce purely manual classification effort.

17. Limitations

- Accuracy depends on photo quality (lighting, angle, occlusion, distance).
- Mixed-breed or heavily cross-bred animals may be ambiguous.
- The model is strong on the breeds it was trained for; rare local variants may be less reliable.
- First model download requires disk space (~320 MB weights).
- Not a veterinary diagnosis tool — it only estimates breed identity.
- Complex farm scenes with multiple animals may confuse the classifier.

18. Future Enhancements

- Mobile application (Android/iOS) for field workers.
- Offline-first packaging for rural areas with weak internet.
- Disease / health symptom detection from images.
- Database logging of predictions (MongoDB/MySQL) for analytics.
- Fine-tuning on additional regional farm photos to raise accuracy further.
- Cloud deployment (for example Vercel frontend + GPU inference backend).
- Multi-animal detection and individual animal ID tracking.

19. Conclusion

PashuPehchaan demonstrates a complete, practical pipeline for **Image Based Breed Recognition for Cattle and Buffaloes of India**. It combines a modern web interface, a FastAPI backend, and a trained ConvNeXt deep-learning model to turn an uploaded photograph into breed identity and useful livestock information within seconds. By replacing slow and error-prone manual guesswork with AI assistance, the system supports better livestock data collection for farmers, students, researchers, and field agencies. The project is easy to run locally with `npm run dev`, works on both Mac and Windows environments, and forms a solid foundation for future mobile and offline extensions.

— End of Document —